applications. These apps can run as console apps or can be installed as Windows services (or as systemd daemons in Linux) to run background tasks or read from message queues, for example.

Kestrel and the web framework of ASP.NET Core build on top of the generic host functionality introduced in ASP.NET Core 3.0. To configure a typical ASP.NET Core app, you configure the generic host features that are common across all apps—features such as configuration, logging, and dependency services. For web applications, you then also configure the services, such as Kestrel, that are necessary to handle web requests. In chapter 34 you'll see how to build applications using the generic host to run scheduled tasks and build background services.

Even in .NET 7, `WebApplication` and `WebApplicationBuilder` use the generic host behind the scenes. You can read more about the evolution of ASP.NET Core's bootstrapping code and the relationship between IHost and WebApplication on my blog at http://mng.bz/gBBv.

Once the configuration of the `IHostBuilder` is complete, the call to `Build` produces the `IHost` instance, but the application still isn't handling HTTP requests yet. It's the call to `Run()` that starts the HTTP server listening. At this point, your application is fully operational and can respond to its first request from a remote browser.

30.3 The Startup class: Configuring your application

As you've seen, `Program` is responsible for configuring a lot of the infrastructure for your app, but you configure some of your app's behavior in `Startup`. The `Startup` class is responsible for configuring two main aspects of your application:

- DI container service registration
- Middleware configuration and mapping of endpoints